

INTERNSHIP REPORT

Evaluation of Approaches for Identification of Dynamic Clusters



**DRDO - Centre for Artificial Intelligence & Robotics
(CAIR)**

Submitted By

Ananya BM

CB.SC.U4CSE23708

Under the Guidance of

Smt. Shyni Thomas

Scientist 'G'

CAIR, DRDO

Amrita Vishwa Vidyapeetham

2026

DECLARATION

I hereby declare that the project report entitled “**Evaluation of Approaches for Identification of Dynamic Clusters**” is a record of original work carried out by me under the supervision and guidance of **Smt. Shyni Thomas, Scientist ‘G’, CAIR, DRDO**. The matter embodied in this report is original and has not been submitted for the award of any other degree or diploma.

To the best of my knowledge and belief, this work contains no material previously published or written by another person, except where due acknowledgement has been made in the text.

I hereby certify that this submission is my own work and that all the results and conclusions presented are a true and accurate reflection of my efforts during the internship period.

Ananya BM

CB.SC.U4CSE23708

Amrita Vishwa Vidyapeetham

ACKNOWLEDGEMENT

I wish to express my sincere gratitude and heartfelt thanks to my guide, **Smt. Shyni Thomas, Scientist 'G', CAIR - DRDO**, for her valuable guidance, continuous encouragement, and support throughout the internship period.

I would also like to thank the Director and the entire team at **CAIR, DRDO** for providing me with the opportunity to work in such a prestigious research environment.

I extend my sincere thanks to the faculty members of **Amrita Vishwa Vidyapeetham** for their encouragement and academic support.

Finally, I would like to thank my family and friends for their constant motivation and encouragement throughout this journey.

ABSTRACT

Real-time clustering of continuously arriving 3D data is a major challenge in modern intelligent systems, especially in applications involving dynamic environments, surveillance systems, IoT devices, anomaly detection, autonomous systems, and spatial monitoring.

Traditional clustering algorithms are primarily designed for static datasets and often struggle to adapt to continuously evolving data distributions. They also face challenges in handling noise efficiently while maintaining computational performance in streaming environments.

This project presents a dynamic 3D stream clustering framework that evaluates multiple clustering approaches for identifying evolving clusters in continuously arriving spatial data. The system focuses on the implementation and comparative analysis of clustering algorithms such as DBSCAN, ACSC (Ant Colony Stream Clustering), and concepts inspired by DenStream for handling dynamic streaming data.

The ACSC-based framework utilizes micro-clustering techniques inspired by ant colony behavior, where incoming points are incrementally assigned to clusters based on Euclidean distance and cluster density properties. Each micro-cluster maintains compact statistical summaries including Linear Sum (LS), Squared Sum (SS), centroid, and radius, enabling efficient updates without storing the complete historical dataset.

The project also explores advanced stream clustering concepts such as fading functions, outlier micro-clusters, promotion of dense clusters, and pruning of weak clusters to improve adaptability in dynamic environments.

An interactive visualization interface was developed using PyQt5 to display evolving 3D clusters, outliers, and clustering statistics in real time. Timestamp-based analysis and performance monitoring were incorporated to compare the behavior of different clustering approaches.

The proposed system demonstrates an effective and scalable solution for dynamic

stream clustering and provides a strong foundation for future research in adaptive clustering, real-time analytics, and intelligent monitoring systems.

Contents

1	INTRODUCTION	6
1.1	Background and Motivation	6
1.2	Problem Statement	7
1.3	Project Objectives	8
2	LITERATURE SURVEY	9
2.1	DBSCAN	9
2.2	Ant Colony Stream Clustering (ACSC)	10
2.3	DenStream	11
3	SYSTEM ARCHITECTURE	13
3.1	Overall System Design	13
3.2	GUI Design	14
4	IMPLEMENTATION	16
4.1	DBSCAN Implementation	16
4.2	ACSC Implementation	16
4.3	DenStream Implementation	17
5	RESULTS AND PERFORMANCE ANALYSIS	18
5.1	Experimental Setup	18
5.2	Cluster Formation Analysis	19
5.3	PerformanceMetrics	19
5.4	Outlier Detection Performance	20
5.5	Cluster Stability Analysis	20
5.6	Execution Time Analysis	21

5.7	Comparative Analysis	21
5.8	Advantages and Limitations	22
5.8.1	DBSCAN	22
5.8.2	ACSC	22
5.8.3	DenStream	23
6	CHALLENGES FACED	24
7	CONCLUSION	25
8	FUTURE WORK	26

Chapter 1

INTRODUCTION

1.1 Background and Motivation

The rapid growth of intelligent systems has led to the continuous generation of streaming data from sources such as IoT devices, surveillance systems, autonomous vehicles, sensor networks, and real-time monitoring platforms. Unlike traditional static datasets, streaming data evolves continuously and requires efficient real-time processing techniques.

Clustering is one of the most important techniques in data analysis and machine learning. It is used to group similar data points together based on their properties and similarity measures.

Traditional clustering algorithms are primarily designed for static datasets where the complete dataset is available before processing begins. However, in streaming environments, data arrives continuously over time. Clusters may evolve dynamically, new clusters may emerge, and old clusters may disappear.

Dynamic clustering introduces additional requirements including:

- Incremental data processing
- Real-time cluster updates
- Efficient memory usage
- Handling evolving data distributions
- Noise and outlier management

Density-based clustering techniques such as DBSCAN are highly effective in detecting arbitrarily shaped clusters and identifying noise. However, DBSCAN is not inherently designed for continuously evolving streams.

To overcome these limitations, stream clustering approaches such as ACSC and Den-Stream introduce concepts like micro-clustering, fading mechanisms, incremental updates, and adaptive cluster maintenance.

This project focuses on studying, implementing, and evaluating different approaches for identifying dynamic clusters in continuously arriving 3D spatial data streams.



Figure 1.1: Example of Continuously Arriving 3D Streaming Data

1.2 Problem Statement

The project addresses the challenge of clustering continuously arriving 3D spatial data in dynamic environments.

Traditional clustering algorithms struggle to process streaming data efficiently because they require repeated processing of the entire dataset whenever new data arrives.

The major challenges addressed include:

- Identifying dense regions in continuously arriving 3D data streams
- Detecting and handling outliers effectively
- Adapting to cluster evolution across timestamps
- Maintaining computational efficiency

- Reducing memory overhead
- Comparing clustering behavior in dynamic environments

1.3 Project Objectives

- To study stream clustering techniques
- To implement DBSCAN as a baseline clustering algorithm
- To implement ACSC for dynamic clustering
- To study DenStream concepts including fading and pruning
- To analyze cluster evolution across timestamps
- To compare clustering algorithms based on adaptability and performance
- To visualize evolving 3D clusters in real time

Chapter 2

LITERATURE SURVEY

2.1 DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is a density-based clustering algorithm that groups nearby points together based on neighborhood density.

The Euclidean distance between two 3D points is calculated as:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$$

Advantages include:

- Detection of arbitrarily shaped clusters
- Effective noise handling
- No need to specify cluster count

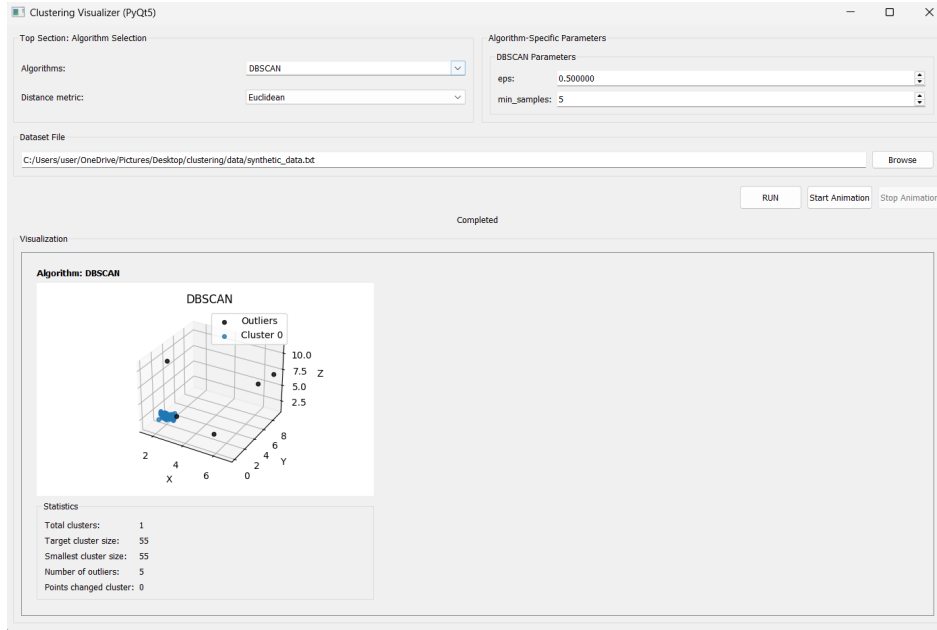


Figure 2.1: DBSCAN Clustering Output

2.2 Ant Colony Stream Clustering (ACSC)

ACSC is inspired by ant colony behavior and utilizes incremental micro-clustering.

Each micro-cluster stores:

- Linear Sum (LS)
- Squared Sum (SS)
- Centroid
- Radius

The centroid is calculated as:

$$Centroid = \frac{LS}{N}$$

The radius is calculated as:

$$r = \sqrt{\frac{SS}{N} - \left(\frac{LS}{N}\right)^2}$$

Advantages include:

- Incremental updates
- Efficient memory usage
- Real-time adaptability

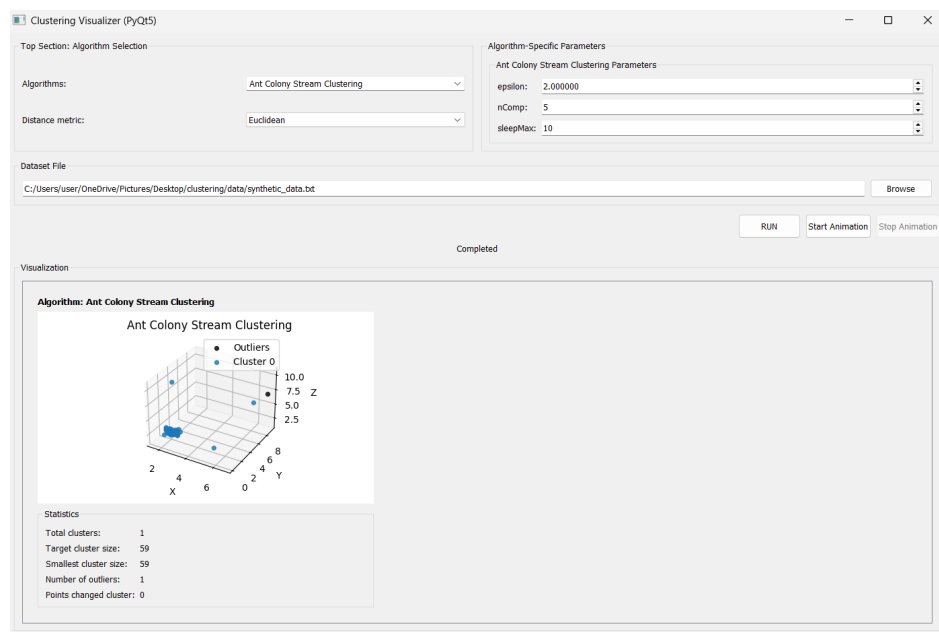


Figure 2.2: ACSC Dynamic Stream Clustering

2.3 DenStream

DenStream is a stream clustering algorithm designed for evolving data streams.

DenStream introduces:

- Potential Micro-Clusters
- Outlier Micro-Clusters
- Fading Functions
- Pruning Mechanisms

The fading function is:

$$f(t) = 2^{-\lambda t}$$

where λ is the decay factor.

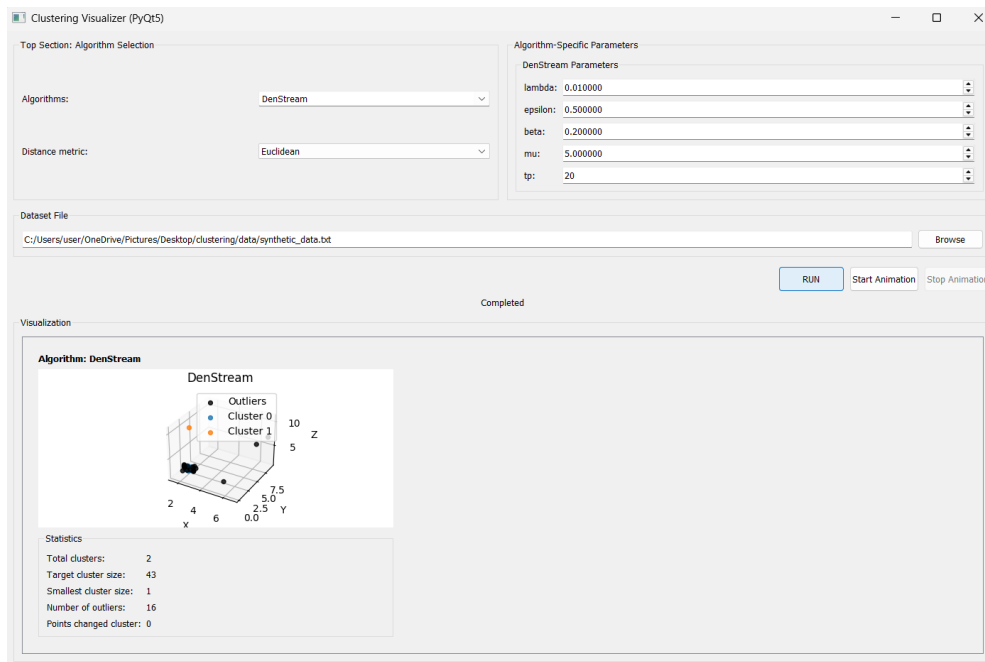


Figure 2.3: DenStream Workflow and Micro-Cluster Maintenance

Chapter 3

SYSTEM ARCHITECTURE

3.1 Overall System Design

The project follows a modular architecture.

The major modules include:

- Synthetic Data Generator
- Clustering Engine
- DBSCAN Module
- ACSC Module
- DenStream Module
- GUI Visualization System
- Performance Evaluation Module

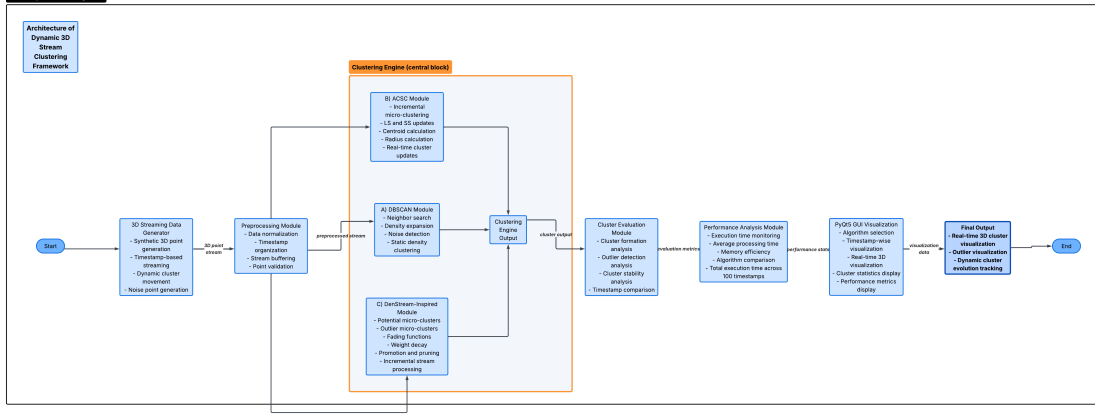


Figure 3.1: Overall System Architecture

The system architecture consists of multiple interconnected modules responsible for synthetic stream generation, preprocessing, clustering, evaluation, and visualization. Incoming 3D streaming data is processed incrementally by the clustering engine, which supports DBSCAN, ACSC, and DenStream-based clustering approaches. The generated clustering results are evaluated and visualized through the PyQt5-based GUI system.

3.2 GUI Design

The graphical user interface was developed using PyQt5.

The GUI supports:

- Algorithm selection
- Timestamp-wise clustering
- Real-time 3D visualization
- Cluster statistics display
- Execution time monitoring

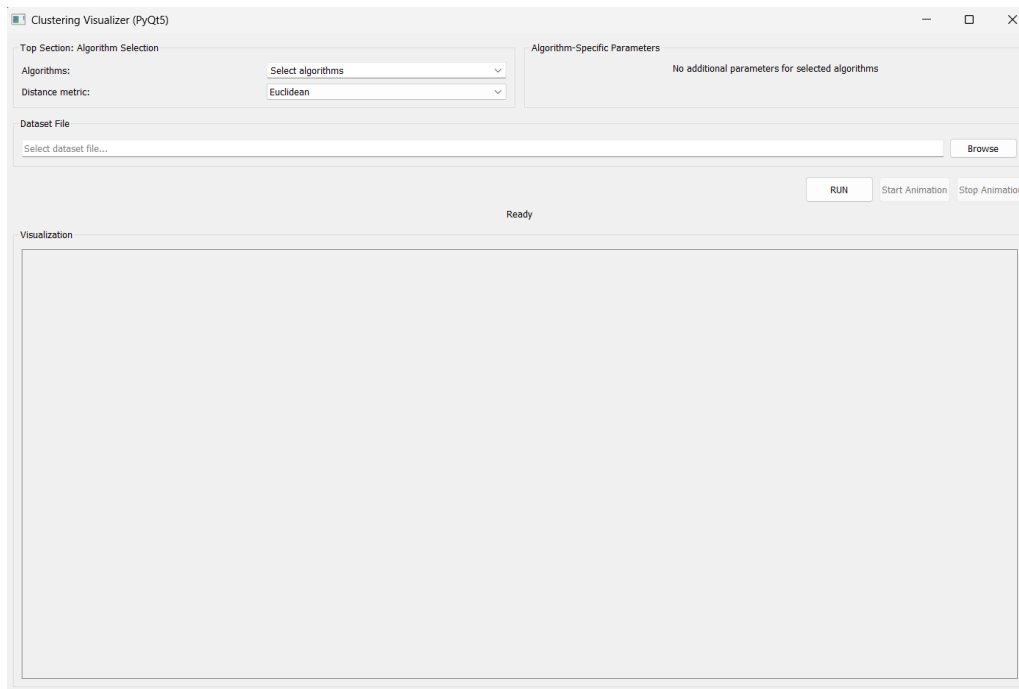


Figure 3.2: PyQt5 GUI for Real-Time Cluster Visualization

Chapter 4

IMPLEMENTATION

4.1 DBSCAN Implementation

DBSCAN was implemented as the baseline clustering algorithm.

The implementation involved:

- Neighbor search
- Cluster expansion
- Density checking
- Noise detection

4.2 ACSC Implementation

The ACSC implementation uses incremental micro-clustering.

Each incoming point follows the workflow:

1. Find nearest cluster
2. Check distance threshold
3. Update LS and SS
4. Update centroid
5. Update radius
6. Create new cluster if required

4.3 DenStream Implementation

A DenStream-based incremental stream clustering engine was implemented for continuously arriving 3D spatial data.

The implementation includes:

- Fading functions
- Weight decay
- Outlier micro-clusters
- Promotion mechanisms
- Pruning of weak clusters

Chapter 5

RESULTS AND PERFORMANCE ANALYSIS

5.1 Experimental Setup

The experiments were conducted on synthetic 3D streaming datasets across 100 timestamps.

For representation and comparison purposes, the detailed results of the first five timestamps are shown in the following tables. The total execution time values correspond to the cumulative processing time across all 100 timestamps.

The algorithms evaluated were:

- DBSCAN
- ACSC
- DenStream

The evaluation focused on:

- Cluster quality
- Outlier handling
- Computational efficiency
- Stability
- Adaptability to evolving distributions

5.2 Cluster Formation Analysis

Timestamp	DBSCAN	ACSC	DenStream
T1	1 cluster (55 pts)	1 cluster (59 pts)	2 clusters (43,1 pts)
T2	1 cluster (35 pts)	1 cluster (40 pts)	5 clusters (max 15 pts)
T3	2 clusters (24,19 pts)	2 clusters (30,20 pts)	5 clusters (max 15 pts)
T4	1 cluster (48 pts)	1 cluster (52 pts)	3 clusters (34,1,1 pts)
T5	1 cluster (40 pts)	2 clusters (42,5 pts)	3 clusters (26,1,1 pts)

5.3 Performance Metrics

The following metrics were used to evaluate the clustering algorithms across streaming timestamps.

Outlier Percentage

The outlier percentage was calculated as:

$$\text{Outlier Percentage} = \frac{\text{Number of Outliers}}{\text{Total Points}} \times 100$$

This metric represents the proportion of points classified as noise or outliers by the clustering algorithm.

Average Execution Time

The average execution time per timestamp was calculated as:

$$\text{Average Execution Time} = \frac{\text{Total Execution Time}}{\text{Number of Timestamps}}$$

where the total execution time corresponds to cumulative processing across all timestamps.

Cluster Stability

Cluster stability was evaluated by measuring the number of points that changed cluster assignments between consecutive timestamps. Lower values indicate more stable clustering behavior over time.

Table 5.1: Cluster Formation Comparison

5.4 Outlier Detection Performance

Timestamp	DBSCAN	ACSC	DenStream
T1	5 (8.3%)	1 (1.7%)	16 (26.7%)
T2	7 (16.7%)	2 (4.8%)	13 (31.0%)
T3	9 (17.3%)	2 (3.8%)	15 (28.8%)
T4	6 (11.1%)	2 (3.7%)	18 (33.3%)
T5	7 (14.9%)	0 (0.0%)	21 (44.7%)
Average	11.7%	2.8%	32.9%

Table 5.2: Outlier Detection Comparison

5.5 Cluster Stability Analysis

Timestamp	DBSCAN	ACSC	DenStream
T1	0	0	0
T2	37	42	30
T3	47	22	13
T4	4	3	10
T5	1	5	13
Total	89	72	66

Table 5.3: Changed Cluster Points Across Timestamps

5.6 Execution Time Analysis

Metric	DBSCAN	ACSC	DenStream
T1	0.002395 s	0.002108 s	0.005667 s
T2	0.001579 s	0.002118 s	0.004788 s
T3	0.001828 s	0.003055 s	0.003204 s
T4	0.002724 s	0.002283 s	0.004127 s
T5	0.002268 s	0.004459 s	0.007439 s
Total (100 timestamps)	0.348 s	0.274 s	0.530 s
Average per Timestamp	0.00348 s	0.00274 s	0.00530 s

Table 5.4: Execution Time Comparison

The timestamp-wise execution times shown above correspond only to the first five timestamps for representation purposes. The total and average execution times were calculated using the complete set of 100 timestamps, which included varying cluster densities and stream behaviors.

5.7 Comparative Analysis

The experimental results revealed several important observations.

- DBSCAN produced cohesive and interpretable clusters but showed higher cluster volatility during concept drift.
- ACSC achieved strong performance in terms of incremental clustering and computational efficiency.
- DenStream maintained continuous micro-cluster maintenance through fading functions and pruning mechanisms, enabling adaptive handling of evolving data streams.
- DenStream demonstrated better adaptability for continuously evolving data streams across large timestamp ranges.
- ACSC demonstrated faster execution performance, while DenStream required comparatively higher computational overhead.

- DBSCAN and ACSC successfully identified dense evolving regions as meaningful clusters.
- DenStream frequently divided dense regions into smaller micro-clusters, resulting in higher outlier detection.
- Across long streaming durations, DenStream provided more stable stream maintenance and adaptive clustering behavior.

5.8 Advantages and Limitations

5.8.1 DBSCAN

Advantages

- Produces meaningful dense clusters
- Strong noise handling
- Easy interpretation
- Effective for static datasets

Limitations

- Not optimized for streaming data
- Requires repeated processing
- Sensitive to parameter selection

5.8.2 ACSC

Advantages

- Incremental clustering
- Efficient memory usage
- Real-time adaptability

- Fast processing speed

Limitations

- Parameter tuning complexity
- Conservative outlier detection

5.8.3 DenStream

Advantages

- Designed specifically for evolving data streams
- Supports fading and pruning mechanisms
- Better long-term cluster stability
- Adaptive to continuously changing data distributions
- Effective micro-cluster maintenance

Limitations

- Higher computational overhead compared to ACSC
- Sensitive parameter tuning
- Increased micro-cluster fragmentation in dense regions
- Higher execution time due to fading and maintenance operations

Chapter 6

CHALLENGES FACED

The following challenges were encountered:

- Selecting optimal clustering parameters
- Managing evolving clusters
- Handling timestamp synchronization
- Efficient memory management
- Real-time visualization updates
- Implementing incremental processing
- Reducing computational overhead

Chapter 7

CONCLUSION

This project explored multiple approaches for identifying dynamic clusters in continuously arriving 3D spatial streams.

DBSCAN served as a strong baseline clustering algorithm and produced cohesive dense clusters with effective noise handling. However, its static nature limited adaptability in evolving streaming environments.

ACSC demonstrated strong performance for streaming applications through incremental micro-clustering techniques. The algorithm provided efficient processing speed, low memory overhead, and effective clustering quality.

DenStream-inspired concepts such as fading functions, micro-cluster maintenance, and pruning mechanisms significantly improved adaptability and stability in evolving streaming environments. The algorithm demonstrated better long-term stream maintenance and adaptive clustering behavior across continuously arriving timestamps.

Although DenStream introduced comparatively higher computational overhead and increased micro-cluster fragmentation, it provided stronger support for evolving stream processing and dynamic cluster management.

Experimental evaluation across 100 timestamps showed that ACSC achieved faster execution performance, while DenStream demonstrated superior adaptability and cluster stability for long-duration streaming data.

The developed framework successfully demonstrated real-time clustering, timestamp-based evolution tracking, and comparative performance evaluation for dynamic stream clustering systems.

Chapter 8

FUTURE WORK

Possible future improvements include:

- Full-scale DenStream optimization
- Adaptive parameter tuning
- GPU acceleration
- Distributed stream clustering
- Real-world IoT sensor integration
- Deep learning assisted clustering
- Hybrid clustering approaches
- Large-scale streaming evaluation

REFERENCES

1. Feng Cao, Martin Ester, Weining Qian, and Aoying Zhou, “Density-Based Clustering over an Evolving Data Stream with Noise,” Proceedings of the SIAM International Conference on Data Mining, 2006.
2. Ant Colony Stream Clustering: A Fast Density Clustering Algorithm for Dynamic Data Streams Conor Fahy, Shengxiang Yang , Senior Member, IEEE, and Mario Gongora
3. Han, J., Kamber, M., & Pei, J. Data Mining: Concepts and Techniques.
4. Research papers on stream clustering and micro-clustering techniques.